

PROJECT REPORT

MP3 Tag Reader, Editor & Album Art Extractor

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Protocol Standard: ID3v2.3 Specification

1. INTRODUCTION

The MP3 Tag Reader and Editor is a comprehensive C-based command-line utility designed to parse, display, and mutate metadata embedded within MP3 audio files. Adhering to the ID3v2.3 specification, this project processes complex binary data structures to extract text metadata (such as Song Title, Artist, Album, and Genre) and binary payloads (Album Art). The project demonstrates proficiency in systems-level programming, strict byte-alignment handling, memory management, and binary file I/O operations.

2. SYSTEM ARCHITECTURE & MODULES

The system is architected into specific functional modules to separate the reading, writing, and raw binary extraction processes.

2.1. ID3 Reader Module (id3_reader.c)

This module reads the MP3 file linearly. It validates the "ID3" file signature and handles text encoding dynamically. Because ID3v2 text frames can be encoded in ISO-8859-1 (ASCII) or UTF-16 with Byte Order Marks (BOM), the reader uses the encoding flag (e.g., `0x00` vs `0x01`) to appropriately downsample wide characters to standard C strings.

2.2. ID3 Writer Module (id3_writer.c)

This module performs in-place binary mutations using the `r+` file mode. It targets specific frames (like `TIT2` for Title) and overwrites the existing data with user input. It enforces strict boundary checks to ensure newly edited tags do not overflow the pre-allocated frame capacity, preventing the corruption of subsequent frames.

2.3. Image Extractor Module (image.c)

This module targets the `APIC` (Attached Picture) frame. It dynamically parses the MIME-type string (e.g., `image/jpeg`) to determine the correct file extension, skips the description bytes, and streams the raw embedded image data into an independent, standalone image file.

3. PROTOCOL COMPLEXITY & CORE LOGIC

Working with the ID3v2 standard requires handling highly specific layout conventions, particularly regarding Endianness and Synchsafe Integers.

Synchsafe Integer Parsing

To prevent audio players from confusing metadata headers with audio synchronization signals, the ID3 global header size is stored as a 32-bit "Synchsafe" integer, where the most significant bit of each byte is zeroed out. The project correctly decodes this using 7-bit arithmetic shifting, as opposed to standard 8-bit shifts used for normal frame sizes.

```
int get_size(FILE *fp) {
    char s[4];
    fread(s, sizeof(char), 4, fp);
    int get = 0;
    // 7-bit shift for Synchsafe integers
    get = (s[0] << 21) | get;
    get = get | (s[1] << 14);
    get = get | (s[2] << 7);
    get = get | (s[3]);
    return get;
}
```

4. KEY FEATURES & CONSTRAINTS

Feature	Implementation Details
Encoding Adaptation	Identifies text encoding flags (<code>0x00</code> , <code>0x01</code> , <code>0x02</code>) and adjusts buffer allocation and <code>fseek</code> skips to correctly interpret ASCII vs. UTF-16 text streams on Little-Endian architectures.
In-Place Editing	Mutates binary data without loading the entire file into memory. Truncates shorter strings by packing the remaining frame buffer with null bytes (<code>0x00</code>).
APIC Extraction	Identifies the MIME type, isolates the binary payload, and extracts album artwork (JPEG/PNG) directly to disk.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

While the tool efficiently handles metadata extraction and basic modification, several areas offer room for advanced optimization:

- **Fixed Frame Size Constraints:** Because edits are performed in-place (`r+` mode), the new tag cannot exceed the character length of the original tag. Modifying the program to rewrite the entire ID3 header dynamically (shifting the audio frames) would allow for unrestricted string lengths.
- **Binary Stream Corruption Risk:** In the current implementation of `get_image_uth` , the code utilizes `fseek(fpr, 1, SEEK_CUR)` to skip alternate bytes when processing UTF-16 text. Applying this logic to raw binary image data (which utilizes all 8 bits per byte) will corrupt the extracted image. This parsing loop must be isolated from the text-handling logic.
- **Memory Safety Checks:** The `create_tag_data` utility allocates memory based on user-provided file inputs. Adding robust constraints against artificially inflated frame-size declarations would harden the utility against malformed or malicious MP3 files.

6. CONCLUSION

The MP3 Tag Reader, Editor, and Extractor project demonstrates a deep understanding of standard binary protocols, memory allocation, and bitwise arithmetic. By navigating the complexities of ID3v2.3—including synchsafe decoding and dynamic encoding interpretation—the utility serves as a highly functional tool and an excellent showcase of systems programming capabilities in C.